LECTURE 21

Scaling and Performance in Backend Engineering

Detailed backend engineering notes • GitHub-ready companion Markdown

Central principle
Measure the system, identify the actual bottleneck, and choose the optimization or scaling strategy that addresses it.

Core idea: Understand how to measure performance, locate the real bottleneck, and choose scaling techniques based on evidence rather than assumptions.

1. What does “fast” mean? — Latency

Latency is the elapsed time between an action/request starting and the result becoming available. In a web application, the user-perceived interaction can include:

- The user clicks a button.
- The browser sends a request over the network.
- The backend receives and processes it.
- The backend may query a database and/or call an external API.
- The backend sends a response, commonly JSON.
- The browser parses/processes the response and renders the result.

The time from the user action to the visible result is the end-to-end experience. Backend engineers often measure request latency at specific boundaries (for example, server receipt to response), so always check what a metric includes.

Latency varies from request to request. A request may be fast because it hits a cache or reaches an idle server; another may be slower because it requires a database query or waits behind concurrent work.

2. Why averages are misleading

A mean latency can hide a small but important group of very slow requests. The lecture illustrates a case where most requests finish quickly but 1% take several seconds. At a million requests per day, 1% still represents 10,000 slow requests.

For performance, inspect the **distribution** rather than relying on one average.

Percentiles

Example: if P99 is 2 seconds, 99% of measured requests completed in 2 seconds or less, while the slowest 1% took at least about 2 seconds. Percentiles are calculated over a defined population/window; they do not mean exactly the same named users are always in the slow group.

Backend teams pay particular attention to P95/P99 because tail latency affects real user experience and can reveal expensive business workflows, complex queries, service synchronization, and external calls.

3. Throughput

Throughput is how much work a system completes per unit of time. For HTTP backends, common units are requests per second (RPS) and requests per minute.

- Latency: how long one request takes.
- Throughput: how many requests the system handles over time.

A system may have low latency at 10 RPS but become much slower as traffic rises. Looking at latency without the corresponding load can therefore be misleading.

Throughput and latency together help answer:

- Can the service handle a Black Friday spike?
- How many concurrent users/requests can it support?
- What happens after an email campaign or media mention?
- At what load does latency begin to rise sharply?

4. Utilization and the latency curve

Utilization is the percentage of a resource's capacity currently in use. Think of a single-server ice-cream shop:

- Empty shop: low utilization; customers are served with little waiting.
- Busy shop: the worker may prepare each ice cream at the same speed, but customers wait in a queue.
- Near capacity: small additional demand can create a disproportionately large queue and waiting time.

The same effect appears in backend systems. Requests wait for CPU, memory, database connections, worker slots, locks, or other constrained resources.

As utilization approaches 100%, latency often rises non-linearly and can grow very sharply. The lecture uses a highway analogy: traffic flows at moderate occupancy, becomes less predictable at high occupancy, and can gridlock when there is no room to move.

Headroom and bursty traffic

Do not plan to operate every resource continuously at 100% utilization. Keep **headroom**—unused capacity that can absorb bursts and variation. The lecture gives 60–80% utilization as a common illustrative operating range, with remaining capacity acting as a buffer; this is not a universal target for every service or resource.

Traffic is bursty, not a perfectly steady metronome. A service whose average load is modest can still be overwhelmed by a sudden spike. Capacity planning must account for peaks, not just averages.

5. Bottlenecks: measure, don't guess

A **bottleneck** is the specific component or step that constrains end-to-end performance.

Do not automatically:

- Add Redis because an API is slow.
- Upgrade PostgreSQL because a database is an obvious suspect.
- Add more servers before checking where time is spent.

Lecture example: remote logging

A product GET endpoint appeared slow. The team first added caching, but the endpoint remained slow. Granular timings then showed:

- Database query: about 10 ms.
- Cache operation: about 5 ms.

- Synchronous remote logging call: about 500 ms.

The actual bottleneck was waiting for the remote logging service—not the database. This illustrates the rule: **measure each meaningful stage before choosing an optimization**.

Other less-obvious sources of latency include JSON/XML serialization, a large response payload and network transfer, external API calls inside loops, and synchronous work that could be moved off the critical request path.

6. Profiling, flame graphs, and distributed tracing

Profiling

Profiling measures where an application spends execution time. A profiler observes a running program and records information about function execution and resource use.

A **flame graph** visualizes call stacks:

- Functions are represented by stacked blocks.
- Wider blocks indicate more sampled time.
- Stacked blocks show call relationships.

Flame graphs help identify expensive code paths, but the lecture notes that conventional CPU profiling is more naturally suited to CPU-bound work than to explaining time spent waiting on I/O.

CPU-bound vs I/O-bound

Many ordinary backend performance issues are I/O-bound. A CPU flame graph alone may not explain long database or network waits.

Distributed tracing

Distributed tracing follows one request as it travels through application components and services. It records spans/timings such as:

- Request arrival.
- Handler/service execution.
- Database query start/end.
- External API call start/end.
- Response completion.

Example from the lecture: business logic might take only 2 ms while a database operation takes 800 ms. Tracing reveals where the end-to-end time is spent and directs investigation to the slow stage.

Practical distinction

- Logs: event records and diagnostic context.
- Metrics: aggregated numerical measurements over time (latency, throughput, errors).
- Profiles: where program execution time/resources are spent.
- Traces: the path and timing of an individual request across components.

7. Database performance: N+1 queries

The **N+1 query problem** occurs when an application performs:

- One query to fetch a collection of N records.
- One additional query for each record to fetch related data.

Example: fetch 20 blog posts, then query the author separately for each post. That is 1 + 20 = 21 queries. As the collection grows, query count grows linearly.

Why it hurts:

- Each query has network/driver/database overhead.
- The database must parse/plan/execute and return each query.
- Repeated round trips add up even when each individual query is fast.

Better approaches

- Fetch related data in bulk.
- Use SQL joins (`JOIN`, `LEFT JOIN`) when appropriate.
- Use ORM eager-loading/bulk-fetch features.
- Django examples mentioned: `select_related` and `prefetch_related`.
- Rails example mentioned: `includes`.
- Inspect generated SQL; ORM-looking code can still issue many queries.

The goal is to avoid issuing one database round trip per item when a bulk query can retrieve the needed relationship data.

8. Database indexes

An **index** is an auxiliary data structure that helps the database locate rows without scanning every row. The lecture uses a library catalog analogy: a catalog points to the shelves where books by a particular author are located.

A sequential/full-table scan examines rows to find matches. A suitable index can let the database navigate directly to relevant entries. The lecture describes B-tree indexes as a common structure and briefly mentions hash indexes.

Index trade-offs

Indexes are not free:

- They consume storage.
- Inserts, updates, and deletes must maintain affected indexes.
- Too many indexes can slow write operations and increase maintenance overhead.

Do not index every column. Choose indexes based on actual query patterns and observed bottlenecks. Primary-key columns are commonly indexed automatically by PostgreSQL; other useful columns may need explicit indexes.

Composite index

A **composite index** covers multiple columns, for example `(user_id, created_at)`, when queries commonly filter/order using that combination. Column order matters: the index is most useful for query patterns that align with its leading columns. Separate single-column indexes are not always equivalent.

Covering index

A **covering index** contains the indexed/search columns plus the additional columns needed by a query, allowing the database in suitable cases to answer from the index without visiting the table heap. Whether an index-only scan is actually possible depends on the database engine, query, and visibility/storage conditions.

Inspect query plans

Use `EXPLAIN` / `EXPLAIN ANALYZE` (PostgreSQL) to inspect how a query is executed:

- Which tables are scanned or joined?
- Is a sequential scan or index scan used?
- Which index is selected?
- Where is the time spent?

Use actual query plans and production observations to guide indexing decisions rather than adding indexes blindly.

9. Database connection overhead and connection pooling

Opening a database connection is more expensive than it may appear in local development. A new connection can involve:

- TCP connection establishment (including the handshake).
- Authentication.
- Database session setup/state.
- Database-side resources allocated for that connection.

Creating a fresh connection for every query adds latency and wastes resources. Databases also have finite connection capacity.

Connection pooling

A **connection pool** maintains reusable database connections. Application code borrows a connection, performs work, and returns it to the pool instead of repeatedly creating and closing connections.

Benefits:

- Avoids repeated connection setup.
- Controls the number of simultaneous database connections.
- Reuses established connections.

Internal vs external pooling

- **Internal/application-level pool:** each application instance maintains its own pool, often through its database driver.
- **External pooler:** a separate service sits between application instances and the database. The lecture names PgBouncer as a popular PostgreSQL pooler.

A key capacity trap: if each of three application instances is configured for up to 150 connections, the combined potential is 450 connections. If the database can handle only 300, autoscaling application instances can exhaust the database connection limit. Pool sizing must be calculated across all instances, not one instance in isolation.

10. Caching as a performance tool

Caching can reduce repeated expensive work. The lecture's example stores the result of an expensive query in Redis or another cache:

- Database-backed request: illustrative 500–800 ms.
- Cache response: illustrative 50 ms.

These figures are examples, not universal benchmarks.

Cache invalidation

When source data changes, an old cached value may become stale. Two broad approaches are discussed:

- **Time-based invalidation / TTL:** cache entries expire after a chosen duration. Simple, but freshness depends on the TTL and access pattern.
- **Event-based invalidation:** when source data changes, the application invalidates/deletes the corresponding cache entry. More direct, but every relevant update path must perform invalidation reliably.

Both approaches involve trade-offs. Cache invalidation and consistency are difficult because cached copies must remain sufficiently aligned with the source of truth.

Where does the cache live?

- **Local/in-process cache:** fast access without a network hop, but every server instance has its own copy. With multiple instances, copies can diverge.
- **External/distributed cache:** shared cache service such as Redis or Memcached. Easier to share across instances, but requires a network round trip.
- **Tiered cache:** small local cache for very hot data backed by a shared distributed cache.

Common caching patterns

The lecture focuses especially on cache-aside and write-through, and briefly references write-behind.

Cache hit rate

Cache hit rate is the proportion of cache lookups served by the cache rather than falling through to the original source.

For example, a 90% hit rate means 90% of lookups are served by cache and 10% miss and need the source. A low hit rate may mean the cache is not helping enough.

Factors discussed:

- TTL and expiry choices.
- Cache size/memory.
- User and endpoint access patterns.
- Which data is actually requested frequently.

A large TTL can improve hit rate but increases the risk of stale data. A small TTL improves freshness but can increase misses and source load.

11. Scaling: vertical vs horizontal

As user traffic grows, a server eventually reaches resource limits. The lecture introduces two broad approaches.

Vertical scaling (scale up)

Increase the resources of one machine:

- More CPU cores.
- More RAM.
- More/faster storage.
- More network capacity.

Advantages

- Simple and intuitive.
- Often requires little or no application-code change.
- Avoids some multi-server operational complexity.
- A larger machine can be more straightforward than coordinating several smaller ones.

Limitations

- Hardware/instance sizes have a ceiling.
- A single machine remains a single point of failure.
- One machine may not provide good geographic proximity to users everywhere.

Horizontal scaling (scale out)

Add more server instances and distribute work among them.

Advantages

- Capacity can grow by adding instances.
- Redundancy: one instance failing need not take down the whole service if the system is designed correctly.
- Can place capacity in multiple regions to reduce user-to-server network distance.

Costs and complexity

- Requests must be distributed, commonly with a load balancer.

- A load-balancing algorithm/strategy must be selected.
- State and updates must be coordinated across instances.
- Network partitions/failures can prevent instances from communicating.
- Failed instances must be detected and removed from routing; recovered instances must be safely reintroduced.
- Distributed systems introduce trade-offs and new failure modes rather than eliminating all problems.

Horizontal scaling is not “free capacity”: it exchanges a single-machine limit for coordination, networking, consistency, routing, and operational complexity.

12. A practical performance investigation workflow

- **Define the symptom.** Which endpoint/workflow is slow? What is the affected time window?
- **Measure user-visible and server-side latency.** Inspect P50/P95/P99, not only the mean.
- **Record the load.** Include RPS, concurrency, resource utilization, and traffic bursts.
- **Trace the request path.** Separate handler, business logic, database, cache, external calls, serialization, and network time.
- **Find the constrained resource.** CPU, memory, connection pool, database query, network, queue, or downstream dependency.
- **Inspect detailed evidence.** Use logs, metrics, traces, profiles, flame graphs, and database query plans as appropriate.
- **Choose a targeted change.** Examples: batch related queries, add a justified index, tune a pool, move non-critical work off the request path, or cache an expensive repeated read.
- **Re-measure under representative load.** Compare the same latency percentiles and throughput before/after.
- **Check trade-offs.** Freshness, write cost, memory, connection limits, failure modes, and operational complexity.
- **Preserve headroom.** Plan for bursts and failures rather than targeting constant 100% utilization.

13. Interview questions and concise answers

1. What is the difference between latency and throughput? Latency is the time taken by an individual request; throughput is the amount of work completed per unit time, such as RPS.

2. Why are percentiles more useful than average latency? Averages can hide a small tail of very slow requests. P95/P99 expose the experience of slower requests.

3. What does P99 = 2 seconds mean? 99% of measured requests completed in 2 seconds or less; the slowest 1% took longer.

4. Why does latency rise sharply near full utilization? Requests compete for constrained resources and queue. Near saturation, small load increases can create disproportionately large waiting times.

5. What is headroom? Reserved unused capacity that absorbs traffic bursts and variability.

6. What is a bottleneck? The component or stage limiting overall system performance. Find it by measuring rather than assuming.

7. Profiling vs distributed tracing? Profiling shows where program execution/resources are spent; tracing follows an individual request across components and shows where its time is spent.

8. What is the N+1 query problem? One collection query followed by one related-data query per item. Solve with joins, eager loading, or bulk fetching.

9. Why not index every database column? Indexes consume storage and add write/maintenance cost. Index according to observed query patterns.

10. What is a composite index? An index spanning multiple columns; column order matters and should match common query predicates/order.

11. Why use a connection pool? To reuse established connections, reduce connection setup overhead, and control concurrent database connections.

12. What can go wrong when autoscaling with per-instance pools? The total possible connections can exceed the database's connection limit.

13. What is cache-aside? Check cache first; on miss read the source and populate the cache. The application manages invalidation.

14. Local cache vs distributed cache? Local cache avoids a network hop but copies can diverge between instances; distributed cache is shared but adds network latency.

15. Vertical vs horizontal scaling? Vertical scaling increases one machine's resources; horizontal scaling adds more machines/instances and distributes requests.

16. Why does horizontal scaling add complexity? It requires load balancing, coordination/state consistency, failure detection, network handling, and operational management.

14. Quick revision sheet

- **Latency** = time for a request/interaction.
- **Throughput** = requests/work per unit time.
- **P50** = median; **P95** = 95% at or below; **P99** = 99% at or below.
- Averages hide tail latency.
- Utilization near 100% can cause queueing and sharp latency growth.
- Keep headroom for bursty traffic.
- **Bottleneck** = actual limiting stage; measure, don't guess.
- **Profiler/flame graph** = function execution hotspots; **trace** = request journey across components.
- **N+1** = 1 collection query + N related queries; batch/join/eager-load.
- Indexes speed suitable reads but cost storage and writes.
- `EXPLAIN ANALYZE` helps inspect actual query execution.
- Connection pools reuse connections; size pools across all app instances.
- Cache invalidation strategies include TTL and event-based invalidation.
- Cache-aside, write-through, and write-behind have different consistency/durability trade-offs.

- **Vertical** = bigger machine; **horizontal** = more machines.
- Horizontal scale adds load balancing, coordination, and failure-handling complexity.

15. Source-faithful scope notes

These notes follow the supplied lecture transcript. The lecture uses illustrative latency, utilization, query-count, and cache examples; treat them as teaching examples rather than universal benchmarks or recommended production thresholds. The transcript introduces the concerns around load balancing, synchronization, network failures, and server health at the end, but does not fully teach particular algorithms or distributed-consistency protocols in this segment.